



ASSIGNMENT TITLE :

General Report on Data Types and OS Memory Management (Lab_4)

Module Name: Operating Systems, Security, and Networks

Intake: March 2025

Submission date: September 29th,2026

Submitted By

Name : Ujwal Acharya
Student ID : 16537235
College ID : 250036

Submitted To

Rupak Rajbanshi

Lecturer

Softwarica College of IT & E-Commerce
In Collaboration with Coventry University
Kathmandu, Nepal

Table of Contents

Table of Figures	3
1. Task 1: Data Type Size Exploration	4
2. Task 2: Memory Segment Address Mapping	6
3. Knowledge check questions:	9

Table of Figures

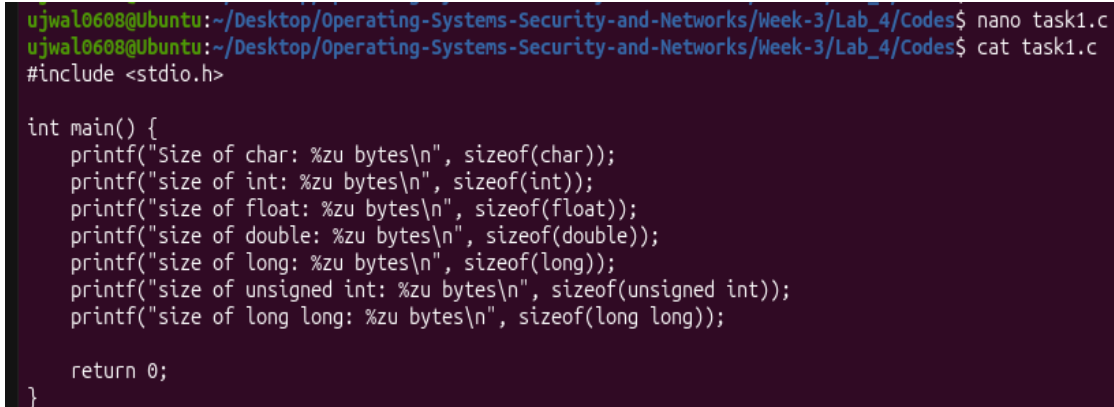
Figure 1: Creating task1.c using nano.....	4
Figure 2: Output of task1.c to verify data type sizes	4
Figure 3: Comparing the data type sizes to the output.....	5
Figure 4: Code for task2.c.....	6
Figure 5: Printing the address of each variable.....	7
Figure 6: Calculating the difference between stack and heap.....	8
Figure 7: Output of task2.c	8

1. Task 1: Data Type Size Exploration

Objective: Use `sizeof()` to observe how much memory each data type uses on your system.

Tasks:

- a) Create a C program that prints the size of: char, int, float, double, long, unsigned int, long long:



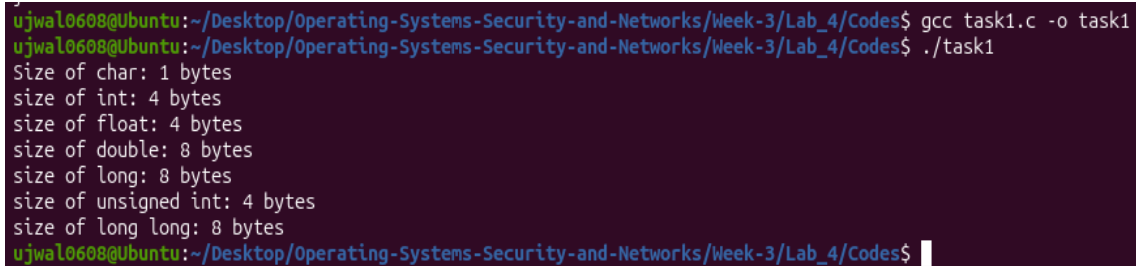
```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ nano task1.c
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ cat task1.c
#include <stdio.h>

int main() {
    printf("size of char: %zu bytes\n", sizeof(char));
    printf("size of int: %zu bytes\n", sizeof(int));
    printf("size of float: %zu bytes\n", sizeof(float));
    printf("size of double: %zu bytes\n", sizeof(double));
    printf("size of long: %zu bytes\n", sizeof(long));
    printf("size of unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("size of long long: %zu bytes\n", sizeof(long long));

    return 0;
}
```

Figure 1: Creating task1.c using nano

- b) Compile and run on your system:



```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ gcc task1.c -o task1
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ ./task1
Size of char: 1 bytes
size of int: 4 bytes
size of float: 4 bytes
size of double: 8 bytes
size of long: 8 bytes
size of unsigned int: 4 bytes
size of long long: 8 bytes
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$
```

Figure 2: Output of task1.c to verify data type sizes

c) Compare the output with Table 1:

TABLE I
DATA TYPE SIZES ACROSS SYSTEM ARCHITECTURES

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

Figure 3: Comparing the data type sizes to the output

In the above Figure 3 and Figure 1, the size of each data type is the same. The size of char in both is 1 bytes, int is 4 bytes; float is 4 bytes; double is 8 bytes; long long is 8 bytes. But sizeof(long) depends on the platform's data model.

d) Why might sizeof(long) differ between 32-bit and 64-bit systems ?

Long can have different sizes on different system architectures and data models. Examples: on many 32-bit systems, long is 4 bytes, while on 64-bit Linux systems, it is usually 8 bytes.

2. Task 2: Memory Segment Address Mapping

Objective: Observe where different variables live in memory and validate the OS memory layout.

Tasks:

1) Write a program that declares variables in each segment:

- i) Global initialized variable (Data segment)
- ii) Global uninitialized variable (BSS segment)
- iii) Local variable (Stack segment)
- iv) Dynamically allocated variable (Heap Segment)

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ nano task2.c
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ cat task2.c
#include <stdio.h>
#include <stdlib.h>

int global_initialized =10;
int global_uninitialized;

int main() {
    int local_variable =20;
    int *heap_variable =malloc(sizeof(int));

    if(heap_variable == NULL) {
        printf("memory allocation failed.\n");
        return 1;
    }

    *heap_variable=30;

    printf("Memory segment address mapping\n");
    printf("#####\n");
    printf("Global initialized (Data) :%p\n", (void *)&global_initialized);
    printf("Global uninitialized (BSS) : %p\n", (void *)&global_uninitialized);

    printf("Local Variable (Stack) :%p\n", (void *)&local_variable);
    printf("Dyanamic Variable (Heap) :%p\n", (void *)&heap_variable);

    long long difference =(char *)&local_variable - (char *)heap_variable;

    printf("\nAddress Difference (Stack - Heap): %lld bytes\n",difference);

    printf("\nNote: Exact addresses vary between program runs and systems.\n");

    free(heap_variable);

    return 0;
}
```

Figure 4: Code for task2.c

2) Print the address of each variable using %p:

```
printf("Memory segment address mapping\n");
printf("#####\n");
printf("Global initialized (Data) :%p\n", (void *)&global_initialized);
printf("Global uninitialized (BSS) : %p\n", (void *)&global_uninitialized);

printf("Local Variable (Stack) :%p\n", (void *)&local_variable);
printf("Dyanamic Variable (Heap) :%p\n", (void *)&heap_variable);

long long difference =(char *)&local_variable - (char *)&heap_variable;

printf("\nAddress Difference (Stack - Heap): %lld bytes\n",difference);
printf("\nNote: Exact addresses vary between program runs and systems.\n");
```

Figure 5: Printing the address of each variable

3) Identify which segment has the highest addresses:

On a Linux 64-bit system, the stack segment has the highest addresses among the four variables observed.

4) Identify which segment has the lowest addresses:

Typically, the Data/BSS segments have lower addresses than the heap and stack. The ordering can vary depending on the system, compiler, and memory layout.

5) Calculate the difference between the stack and heap:

```
long long difference =(char *)&local_variable - (char *)heap_variable;
printf("\nAddress Difference (Stack - Heap): %lld bytes\n",difference);
```

Figure 6: Calculating the difference between stack and heap

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ gcc task2.c -o task2
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$ ./task2
Memory segment address mapping
#####
Global initialized (Date) :0x6348cbbad010
Global uninitialized (BSS) : 0x6348cbbad018
Local Variable (Stack) :0x7ffc9b090cf4
Dyanamic Variable (Heap) :0x7ffc9b090cf8

Address Difference (Stack - Heap): 31558519631076 bytes

Note: Exact addresses vary between program runs and systems.
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_4/Codes$
```

Figure 7: Output of task2.c

This address difference between the stack and heap was calculated by subtracting the heap address from the stack address. The difference depends on the memory layout of the system and may change between program executions.

6) Observation :

The program showed that different types of variables are stored in different memory segments. The global initialized and uninitialized variables are located in the Data and BSS segments, while local variables are stored on the stack and dynamically allocated variables are stored on the heap. The stack generally has a higher memory address than the heap on the Linux system used.

3. Knowledge check questions:

1) What is a data type in the context of programming and operating systems?

A data type defines the kind of data a variable can store, such as an integer, character, or floating-point number. It also determines how much memory is needed to store that data.

2) Name the five main memory segments in a process memory layout.

The five main memory segments are:

- I) **Text segment:** contains the program's instructions.
- II) **Data segment:** Contains initialized global and static variables.
- III) **BSS segment:** contains uninitialized or zero-initialized global and static variables.
- IV) **Heap:** used for dynamically allocated memory
- V) **Stack:** stores local variables, function parameters, and function calls.

3) What is the difference between the data segment and the BSS segment?

The data segment stores global and static variables that have been explicitly initialized, while the BSS segment stores uninitialized global and static variables. Example: `int a= 5` is in the data segment, and `int b` is in the BSS segment.

4) Why does the OS place the stack at high virtual addresses and the text segments at low addresses?

This layout provides separate regions for different types of memory and allows the stack to grow downward and the heap to grow upward, giving them room to expand toward each other. It is a conventional virtual-memory layout rather than a requirement that every operating system must follow.

5) What happens to stack memory when a function returns? Who reclaims it?

When a function returns, its stack frame is removed, so its local variables and function-call information are no longer valid. The compiler-generated function code adjusts the stack pointer to reclaim that space. The OS does not normally reclaim each stack variable.

6) How does the OS know which memory belongs to which process?

Each process has its own virtual address space and page tables. The CPU's memory-management unit (MMU), using the process's page-table information, translates virtual addresses to physical memory and ensures that one process normally cannot access another process's memory.